

```
%prefijoHasta(?X, +L, ?Prefijo)
prefijoHasta(X, L, Prefijo) :- append(Prefijo, [X | _], L).
```

Si L no está instanciada, no hay manera de instanciarla, ya que el segundo argumento del append tiene un sufijo sin instanciar, y el tercero es la misma L que no está instanciada.
Si L está instanciada, tanto X como Prefijo pueden estar o no instanciados, porque append se encarga de instanciarlos si no lo están, o de verificar que coincidan con un prefijo de L y el elemento siguiente si ya están instanciados.

```
%desde(+X, -Y)
desde(X, X).
desde(X, Y) :- desde(X, Z), Y is Z + 1.
```

Si X no está instanciada, desde(X,Y) va a arrojar el resultado $Y = X$, y luego al entrar en la segunda cláusula va a arrojar un error al intentar realizar una operación aritmética sobre Z sin instanciar.
Si Y está instanciada, va a tener éxito si $Y \geq X$, pero luego (o siempre, si $Y < X$) se va a colgar porque va a seguir generando infinitos valores para Z y comparando sus sucesores con Y, lo cual nunca va a tener éxito.

```
%desde2(+X,?Y)
desde2(X,X).
desde2(X,Y) :- nonvar(Y), X < Y.
desde2(X,Y) :- var(Y), desde2(X,Z), Y is Z + 1.
```

X debe estar instanciada por el mismo motivo que en desde/2.
Si Y no está instanciada, instancia Y en X para el primer resultado, y luego entra por la tercera cláusula y va generando infinitos valores para Y.
Si Y está instanciada, entra por la primera cláusula si es igual a X, y por la segunda en caso contrario, haciendo una comparación entre dos variables ya instanciadas, lo cual funciona correctamente.

```
%preorder(+A, ?L)
preorder(Nil, []).
preorder(Bin(I, R, D),[R | L]) :- preorder(I, LI), preorder(D, LD), append(LI, LD, L).
```

Si A no está instanciado, funciona para el caso $L = []$, porque solo unifica con la primera cláusula. Pero para L no vacía o no instanciada va a entrar (eventualmente) en la segunda cláusula, y va a llamar a preorder con dos variables sin instanciar, lo cual solo le va a permitir generar árboles Nil y listas vacías para luego volver a llamarse infinitamente con argumentos sin instanciar.
Si A está instanciado y L no, instancia L con [] si A es Nil, y en caso contrario calcula los respectivos preorders con I y D ya instanciados y los junta con append.
Si ambos argumentos están instanciados, utiliza unificación y/o append para verificar que L coincida con el preorder de A.

```
%sumlist(+Lista, -Suma) --> Puede ser sumlist(+Lista, ?Suma)
sumlist([], 0).
sumlist([X|XS], S) :- sumlist(XS, N), S is N+X.
```

Este predicado no es reversible en la lista, ya que sus elementos se ponen a la derecha de un is ($S \text{ is } N+X$) y eso da error si la expresión de la derecha no está completamente instanciada.
Por otro lado, si bien está pensado para que la suma venga sin instanciar, sí es reversible en la suma: si la lista es vacía unifica la suma con 0 y da True si es 0 y False si no. Si no lo es, la recursión calcula la suma de la cola y luego el $S \text{ is } N+X$ verifica que la suma sea la correcta.

```
%parMenorQue(+Cota,-N) --> Puede ser parMenorQue(+Cota,?N)
parMenorQue(X, N) :- Z is X-1, between(0,Z,N), 0 =:= mod(N,2).
```

La cota claramente no es reversible, porque está a la derecha de un is, pero N sí lo es porque between es reversible en su tercer parámetro, y a partir de ahí ya queda instanciado sin importar cómo haya venido.